



Middle East Technical University



Department of Computer Engineering

CENG 336

Introduction to Embedded Systems Development THE2

Due: 19nd of April 2026, 23:55

Submission: via **ODTUClass**

1 Introduction

This group assignment will familiarize you with the basic interrupt operations and seven-segment display usage using PIC18F8722 microcontroller and the provided boards. To accomplish this, you are going to implement a simple Flappy Bird game. (Here is the link for the original game that can be played on a browser : <https://flappybird.io/>)

The game will be held over the LED area of ports PORTC-F. An illustration of this is given in Figure 1. The user will provide commands to flap the bird and try to avoid the pipes. Details of the rules and input specifications will be given below.

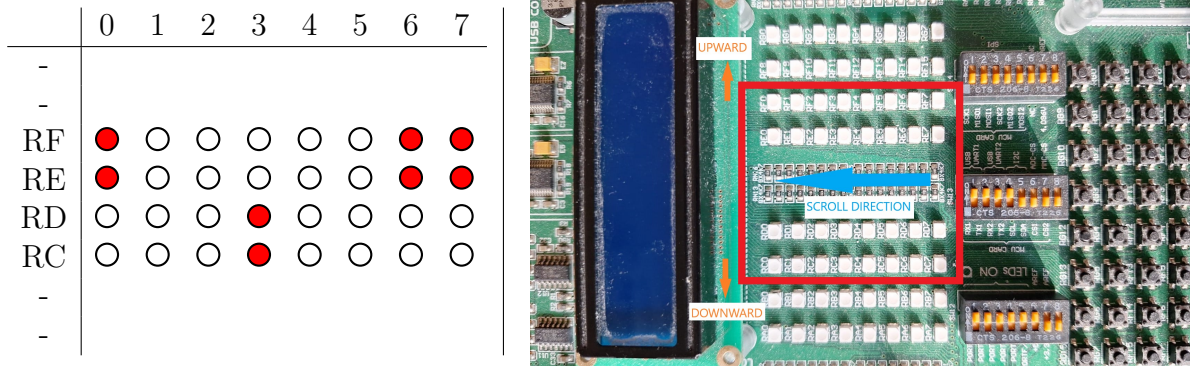


Figure 1: LED board example and area on the actual board.

Finally, the game logic should display the score, which is the sum of the score related to the number of pipes the bird has passed through and the score related to the time the bird has been alive. This will be done using the 7-segment display. The game will end when the bird hits a pipe.

2 Specifications

2.1 Configuration

S-1 The assignment will be written for **PIC18F8722** processor.

- S-2 The processor's clock speed will be **40 MHz**.
- S-3 The assignment will be written in C.
- S-4 The upward direction is from $PORTC$ to $PORTF$, the downward is vice versa.
- S-5 The top of the scene is $PORTF$ and the bottom of the scene is $PORTC$.
- S-6 The scene will be scrolled from rightmost bits to the leftmost bits of the specified ports, as visually described in Figure 1. The rightmost bits are the most significant bits, and the leftmost bits are the least significant bits.

2.2 Inputs

- S-7 The bird will flap its wings via user inputs from $PORTB_0$. $PORTB_0$ **will be triggered by interrupts**.
- S-8 The scrolling step time of the scene will increase via user inputs from $PORTB_1$. $PORTB_1$ **will be triggered by interrupts**.
- S-9 The scrolling step time of the scene will decrease via user inputs from $PORTB_2$. $PORTB_2$ **will be triggered by interrupts**.

2.3 Outputs

- S-10 $PORTC-F$ will be used as outputs. These LEDs define the game area.
- S-11 $PORTH$ and $PORTJ$ will be used to drive the 7-segment display. (Please see Section 3).

2.4 Interrupts

- S-12 As stated above, $PORTB_0$, $PORTB_1$, and $PORTB_2$ will be interrupt driven. Specifically **INT0**, **INT1**, and **INT2** interrupts.
- S-13 Additionally, **TIMER** interrupt will be utilized.
 - **TIMER** interrupt will be used to “blink” the bird (Its LEDs will be on and off).
 - Additionally, this timer will be utilized for bird movement by gravity pull (from $PORTF$ to $PORTC$) and scene scrolling. This movement of the bird is in addition to the user's input (via $PORTB_0$). The scene will scroll from right to left (from $PORTX_7$ to $PORTX_0$, where X is $\{C, D, E, F\}$).

2.5 Initialization

- S-14 All LEDs should be cleared.
- S-15 Interrupt and I-O-related chip states should be set.
- S-16 7-segment display should show zeroes.
- S-17 `SCROLL_STEP_TIME` should be 1000ms.
- S-18 The system should wait **one second**, enable interrupts, and start the game loop.

2.6 Scene Array

```
const char SCENE_ARRAY[4][MAP_SIZE] = {  
{1, 0, 0, 0, 1, 1, ..., 1, 1, 0, 0}, // Represents PORTF  
{1, 0, 0, 0, 1, 1, ..., 1, 1, 0, 0}, // Represents PORTE  
{0, 0, 1, 0, 0, 0, ..., 0, 0, 0, 1}, // Represents PORTD  
{0, 0, 1, 0, 0, 0, ..., 0, 0, 0, 1} // Represents PORTC  
};
```

- S-19 The system uses a 2D constant array `SCREEN_ARRAY[4][MAP_SIZE]` to store the game map, where each row represents a specific hardware port (`PORTF`, `PORTE`, `PORTD`, and `PORTC`).
- S-20 Each column i represents a vertical slice of data, allowing the program to read a quadruple of bits for all four ports at the same time.
- S-21 During every scroll step, the index i increases to fetch the next set of values from the array.
- S-22 A quadruple may have 3 different values. $(1,1,0,0)$, $(0,0,1,1)$ or $(0,0,0,0)$.
- S-23 If a quadruple has 1s, it is considered as **a pipe**.

2.7 Game Loop

- S-24 The game starts with the bird at the second row of the leftmost column from the bottom (Specifically, `PORTD0`).
- S-25 The game starts with the first quadruple of the scene array loaded at the rightmost column (`PORTC7`, `PORTD7`, `PORTE7` and `PORTF7`).
- S-26 The bird will be blinking every **50ms**. **The first 50ms the bird will be on.** Next 50ms, it will be off, and so on.
- S-27 The scene will be scrolling every `SCROLL_STEP_TIME`. `PORTXi+1` value will be shifted to `PORTXi`, for X in $\{C, D, E, F\}$ and i in $\{0, 1, 2, 3, 4, 5, 6\}$. New values of `PORTC7`, `PORTD7`, `PORTE7` and `PORTF7` will be loaded from the next quadruple of the scene array.
- S-28 The bird will be moving down 1 step (e.g. from `PORTF0` to `PORTE0`) every **750ms**. If the bird is already at `PORTC0`, it will not move.
- S-29 At every time the bird passes a pipe (occurs only when scrolling the screen, the quadruple of `(PORTF0, PORTE0, PORTD0, PORTC0)` must form a pipe (defined above) just before scrolling the screen), the score will be incremented by **2**. 7-Segment display will be updated accordingly.
- S-30 At every **2 seconds**, the score will be incremented by **3** due to the bird's alive time. 7-Segment display will be updated accordingly.
- S-31 Whenever the user provides an input via `PORTB0`, the bird will be flapped, meaning that the bird will immediately move up 1 step (e.g. from `PORTD0` to `PORTE0`). If the bird is already at `PORTF0`, it will not move.

- S-32** Whenever the user provides an input via $PORTB_1$, the scrolling step time will be increased by **250ms**. Meaning that $SCROLL_STEP_TIME + = 250ms$, and $SCROLL_STEP_TIME \leq 1500ms$ always.
- S-33** Whenever the user provides an input via $PORTB_2$, the scrolling step time will be decreased by **250ms**. Meaning that $SCROLL_STEP_TIME - = 250ms$, and $SCROLL_STEP_TIME \geq 250ms$ always.
- S-34** When the scrolling step time is changed to a new value, if the necessary time for the current value has already passed from the previous scroll step, then the scene should be scrolled immediately.
- S-35** Whenever the latest quadruple of the scene array is loaded to $PORTC_7$, $PORTD_7$, $PORTE_7$ and $PORTF_7$, the next quadruple to be loaded will be the first quadruple of the scene array, meaning that the scene will be loaded forever.
- S-36** At any time, if the bird collides with a pipe, then it means the bird hits a pipe and the game ends.
- S-37** When a game ends, everything must be frozen for 4 seconds. (No game-logic, no blink etc.) After 4 seconds, the game should restart from the initialization phase described above.

3 7-Segment Display

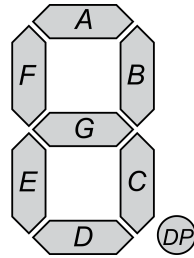


Figure 2: 7-segment display segment naming.

There are three common cathode 7-Segment displays mounted on the development board. $PORTH$ and $PORTJ$ are hooked to this 7-segment display. Specifications are as follows:

- $PORTH_0$ is connected to D_3
- $PORTH_1$ is connected to D_2
- $PORTH_2$ is connected to D_1
- $PORTH_3$ is connected to D_0
- D_0 is the rightmost 7-segment display. (Please check your board. This representation assumes I-O boards of the boards towards up).

- Given these ports above, any data in $\text{PORT } J$ will be sent to 7-segment displays where their enable bit is one. For example, if only D_0 's bit is enabled ($\text{PORT } H_3$ is one), only D_0 will receive the $\text{PORT } J$'s value.
- $\text{PORT } J$ represents the segments on the display:
 - $\text{PORT } J_0$ is connected to A
 - $\text{PORT } J_1$ is connected to B
 - $\text{PORT } J_2$ is connected to C
 - $\text{PORT } J_3$ is connected to D
 - $\text{PORT } J_4$ is connected to E
 - $\text{PORT } J_5$ is connected to F
 - $\text{PORT } J_6$ is connected to G
 - $\text{PORT } J_7$ is connected to DP
- We will not use DP portion of the 7-segment display.
- You can find the diagram on [ODTUClass](#) (Page 2).

4 Example Runtimes

In this section, we will provide two example runtimes for the programs. Table 1 shows the initialization, gravity pulls and scene scrolling with different step times. The second example (Table 2) is from the middle of a game. The user tries to keep the bird alive and earn score. The table timings are somewhat arbitrary since the user may not provide millisecond-correct inputs.

Table 1: Example runtime of the program.

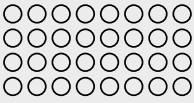
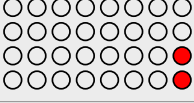
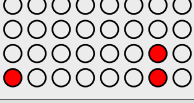
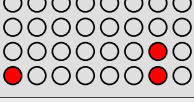
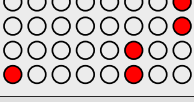
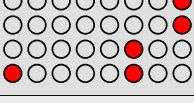
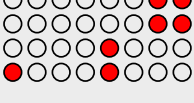
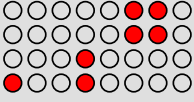
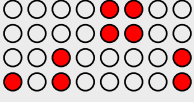
Line No	Game Area	7-SD	Time (ms)	Description
1		0000	0ms	Initial State.
2		0000	+1000ms	The game is started, first scene quadruple is loaded and the bird LED is ON.
3		0000	+50ms	The bird LED is OFF.
4		0000	+50ms	The bird LED is ON.
5		0000	+650ms	Gravity pull occurs but the bird LED is OFF.
6		0000	+50ms	The bird LED is ON.
7		0000	+200ms	The scene has just scrolled.
8		0000	+500ms	Gravity pull occurs but the bird is not affected.
9		0003	+500ms	The scene has just scrolled and score is incremented by 3 because we survived 2 seconds.
10		0003	+600ms	Gravity pull occurred at (+250ms), but the bird wasn't affected.
11		0003	+30ms	The user has provided two inputs for decreasing the scroll step time in a short time. Because new scroll step time (500ms) is already passed, we immediately scroll the screen.
12		0003	+370ms	To align the scroll steps, we scrolled the scene. Also gravity pull occurred.
13		0003	+500ms	The screen has scrolled.

Table 2: Advanced example. Continued from the previous example.

Line No	Game Area	7-SD	Time (ms)	Description
	...			The game continues, total of 3500ms has passed after the start of the game.
14		0003	+5ms	The user provided two inputs for flapping wings.
15		0003	+5ms	The user provided two inputs for flapping wings. However, the second one didn't affect.
16		0003	+290ms	Gravity pull occurred at +240ms.
17		0006	+200ms	The scene has just scrolled, we get score for being alive another 2 seconds.
18		0006	+10ms	The user provided an input for flapping wings.
19		0006	+490ms	The scene has just scrolled, also a gravity pull occurred.
20		0008	+500ms	The scene has just scrolled, the score is incremented by 2 due to passing a pipe.
21		0008	+300ms	Gravity pull occurred at +250ms.
22		0008	+200ms	The scene has just scrolled, but the score isn't incremented because we haven't passed a pipe.
23		0013	+500ms	Gravity pull occurred, the scene has just scrolled, the score is incremented by 5 due to being alive another 2 seconds and passing a pipe.
24		0015	+800ms	The scene scrolled at +500ms, the score is incremented by 2 due to passing a pipe. Gravity pull occurred at +750ms but the bird wasn't affected.
25		0015	+200ms	The scene has just scrolled. However, the bird collides with a pipe at $PORTC_0$. The game is over.

5 FAQ

1. **Q:** Should 7-segment display code be derived by interrupt or in a round-robin fashion? How should we derive multiple 7-segment displays simultaneously?

A: This is up to you. It can be done in both ways. It should be noted that the displayed numbers should not flicker. 7-segment displays should be driven in a round-robin fashion (via either interrupt or in the main loop). Thus, you should send data to one 7-segment display and wait. Then send data to the other 7-segment displays, etc.

2. **Q: In what order should we check and process events (blink, gravity, scrolling) when multiple timer events coincide?**

A: You are free to implement however you want as long as you do not violate the game requirements. Provided the timing and game logic remain consistent with the specifications, the specific execution order of these concurrent events is a design choice.

3. **Q: Which TIMER should we use for timer interrupts?**

A: You are free to use any of the **TIMERs**. In addition, you are free to use multiple **TIMERs** as well, it's a design choice.

6 Grading

The grading will be done manually. Your submission will be compiled, uploaded to a board, and tested by hand. The grading criteria are given below:

- Initialization of the game
- Flapping the wings
- Increasing/decreasing the `SCROLL_STEP_TIME`
- Blinking of the bird
- Gravity pull
- Scene scrolling
- Scoring
- Out-of-bounds movement check
- 7-segment display
- Collision check
- End-game

These conditions may be checked on multiple hardware resets. The exact rubric has not been finalized and thus has not been provided. All of the conditions above will be tested and graded in an as isolated fashion as possible.

7 Regulations

1. The assignment will be done **in groups**.
2. Please track the [THE2 Forum](#) for discussions and questions about this assignment. Other students may have already asked your question, which may be answered already.
3. If your issue is related to *your code* please either e-mail us yalciner@ceng.metu.edu.tr and kursad@ceng.metu.edu.tr (Please CC both of us.) or visit us at our offices: A210 or A409.
4. Please consult your lecture notes and datasheets before asking any questions.
5. **Submission:** Please upload the filled student pack as a single *.zip file. It should be in a compilable state.
6. **Late Policy:** You can extend the deadline by **3 days maximum**. Each day you extend the deadline, **10 points** will be deducted from your grade.
7. **Board Usage:** Since a portion of the grading will be done on physical boards, your solution should work on the boards to get the full grade.
8. There are plenty of resources available to you in ODTUClass. To refer to the instructions and specifications of PIC18F8722, you should use the PIC18F8722 Datasheet. Since you will be using MPLAB X IDE simulation environment for this assignment, Recitation 2 documents can also be a great setup guide. If you set up a local environment, you should use MPLAB X IDE version 5.45 and XC8 Compiler version 2.30; both are available in the [downloads archive](#). You can use the department labs if you cannot set up a local environment.